labsheet=10 PRIYANSHU SHARMA CU240250980 SECTION(B) (47) Mastery in Data Analytics and Visualizations and Career Advancement (Technical)

```
# 1. IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import chi2_contingency

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score
)

import warnings
warnings.filterwarnings("ignore")
```

```
# ============================================================
# 2. LOAD DATASET
# ============================================================

import pandas as pd

file_path = "/content/churn_prediction_results.csv"

print("\n============================================================")
print("EXPERIMENT NO. 10")
print("END-TO-END DATA ANALYTICS PROJECT")
print("CUSTOMER CHURN ANALYSIS")
print("============================================================")
print("PRIYANSHU SHARMA CU240250980")

print("\nLoading dataset...")

try:
    df = pd.read_csv(file_path)

    print("\nDataset loaded successfully!")

    print("\nDataset Shape:")
    print(df.shape)

    print("\nFirst 5 Rows:")
    print(df.head())

    print("\nColumn Names:")
    print(df.columns.tolist())

except FileNotFoundError:
    print("\nERROR: Dataset file was not found.")
    print("\nPlease upload the file:")
    print("churn_prediction_results.csv")

except Exception as e:
    print("\nERROR while loading dataset:")
    print(e)
```

```
============================================================
EXPERIMENT NO. 10
END-TO-END DATA ANALYTICS PROJECT
CUSTOMER CHURN ANALYSIS
============================================================
PRIYANSHU SHARMA CU240250980

Loading dataset...

Dataset loaded successfully!
```

```
Dataset Shape:
(1407, 2)

First 5 Rows:
   Actual_Churn  Predicted_Churn
0         False            False
1         False             True
2         False            False
3          True            False
4         False            False

Column Names:
['Actual_Churn', 'Predicted_Churn']
```

```
# 3. DISPLAY FIRST FIVE ROWS
# ============================================================

print("\n============================================================")
print("FIRST FIVE ROWS")
print("============================================================")
print("PRIYANSHU SHARMA CU240250980")

print(df.head())
```

```
============================================================
FIRST FIVE ROWS
============================================================
PRIYANSHU SHARMA CU240250980
   Actual_Churn  Predicted_Churn
0         False            False
1         False             True
2         False            False
3          True            False
4         False            False
```

```
# 4. DATASET INFORMATION
# ============================================================

print("\n============================================================")
print("DATASET INFORMATION")
print("============================================================")

print("\nNumber of rows and columns:")
print(df.shape)

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)
```

```
============================================================
DATASET INFORMATION
============================================================

Number of rows and columns:
(1407, 2)

Column names:
['Actual_Churn', 'Predicted_Churn']

Data types:
Actual_Churn       bool
Predicted_Churn    bool
dtype: object
```

```
# 5. CHECK MISSING VALUES
# ============================================================

print("\n============================================================")
print("MISSING VALUES")
print("============================================================")

missing_values = df.isnull().sum()

print(missing_values)

total_missing = missing_values.sum()

if total_missing == 0:
    print("\nNo missing values found.")
```

```
else:
    print("\nMissing values found.")
    print("Total missing values:", total_missing)
```

```
============================================================
MISSING VALUES
============================================================
Actual_Churn       0
Predicted_Churn    0
dtype: int64

No missing values found.
```

```
# 6. HANDLE MISSING VALUES
# ============================================================

print("\n============================================================")
print("HANDLING MISSING VALUES")
print("============================================================")

for column in df.columns:

    if df[column].isnull().sum() > 0:

        if df[column].dtype == "object":

            mode_value = df[column].mode()

            if len(mode_value) > 0:
                df[column] = df[column].fillna(mode_value[0])

        else:

            median_value = df[column].median()

            df[column] = df[column].fillna(median_value)

print("Missing values handled successfully.")

print("\nMissing values after cleaning:")
print(df.isnull().sum())
```

```
============================================================
HANDLING MISSING VALUES
============================================================
Missing values handled successfully.

Missing values after cleaning:
Actual_Churn       0
Predicted_Churn    0
dtype: int64
```

```
# 7. CHECK DUPLICATE VALUES
# ============================================================

print("\n============================================================")
print("DUPLICATE VALUES")
print("============================================================")

duplicate_count = df.duplicated().sum()

print("Number of duplicate rows:", duplicate_count)

if duplicate_count > 0:

    df = df.drop_duplicates()

    print("Duplicate rows removed.")

else:

    print("No duplicate rows found.")
```

```
============================================================
DUPLICATE VALUES
============================================================
```

```
Number of duplicate rows: 1403
Duplicate rows removed.
```

```
# 8. DESCRIPTIVE STATISTICS
# ============================================================

print("\n============================================================")
print("DESCRIPTIVE STATISTICS")
print("============================================================")

print(df.describe(include="all"))
```

```
============================================================
DESCRIPTIVE STATISTICS
============================================================
       Actual_Churn Predicted_Churn
count             4               4
unique            2               2
top           False           False
freq              2               2
```

```
# ============================================================
# 9. IDENTIFY NUMERIC AND CATEGORICAL COLUMNS
# ============================================================

import numpy as np

numeric_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

categorical_columns = df.select_dtypes(
    include="object"
).columns.tolist()

print("\n============================================================")
print("NUMERIC COLUMNS")
print("============================================================")

print(numeric_columns)

print("\n============================================================")
print("CATEGORICAL COLUMNS")
print("============================================================")

print(categorical_columns)
```

```
============================================================
NUMERIC COLUMNS
============================================================
[]

============================================================
CATEGORICAL COLUMNS
============================================================
[]
```

```
# 10. FIND ACTUAL CHURN COLUMN
# ============================================================

actual_churn_column = None

actual_candidates = [
    "Actual_Churn",
    "Actual Churn",
    "ActualChurn",
    "actual_churn",
    "actual",
    "Churn",
    "churn"
]

for column in actual_candidates:

    if column in df.columns:

        actual_churn_column = column
        break
```

```
# Automatic search
if actual_churn_column is None:

    for column in df.columns:

        column_name = column.lower()

        if (
            "churn" in column_name
            and "predict" not in column_name
            and "prob" not in column_name
        ):

            actual_churn_column = column
            break
```

```
# 11. FIND PREDICTED CHURN COLUMN
# ============================================================

predicted_churn_column = None

predicted_candidates = [
    "Predicted_Churn",
    "Predicted Churn",
    "PredictedChurn",
    "predicted_churn",
    "prediction",
    "Prediction",
    "Predicted",
    "predicted"
]

for column in predicted_candidates:

    if column in df.columns:

        predicted_churn_column = column
        break


# Automatic search
if predicted_churn_column is None:

    for column in df.columns:

        column_name = column.lower()

        if (
            "predict" in column_name
            and "prob" not in column_name
        ):

            predicted_churn_column = column
            break
```

```
#  12. FIND CHURN PROBABILITY COLUMN
# ============================================================

probability_column = None

probability_candidates = [
    "Churn_Probability",
    "Churn Probability",
    "ChurnProbability",
    "Probability",
    "probability",
    "Predicted_Probability",
    "Predicted Probability",
    "churn_probability"
]

for column in probability_candidates:

    if column in df.columns:

        probability_column = column
        break
```

```
# Automatic search
if probability_column is None:

    for column in df.columns:

        column_name = column.lower()

        if (
            "prob" in column_name
            and "churn" in column_name
        ):

            probability_column = column
            break
```

```
# 13. DISPLAY DETECTED CHURN COLUMNS
# ============================================================

print("\n============================================================")
print("CHURN RELATED COLUMNS")
print("============================================================")

print("Actual Churn Column:",
      actual_churn_column)

print("Predicted Churn Column:",
      predicted_churn_column)

print("Probability Column:",
      probability_column)
```

```
============================================================
CHURN RELATED COLUMNS
============================================================
Actual Churn Column: Actual_Churn
Predicted Churn Column: Predicted_Churn
Probability Column: None
```

```
# 15. PREDICTED CHURN DISTRIBUTION
# ============================================================

if predicted_churn_column is not None:

    print("\n============================================================")
    print("PREDICTED CHURN DISTRIBUTION")
    print("============================================================")

    print(
        df[predicted_churn_column].value_counts()
    )

    plt.figure(figsize=(8, 5))

    sns.countplot(
        x=df[predicted_churn_column]
    )

    plt.title("Predicted Customer Churn Distribution")
    plt.xlabel("Predicted Churn")
    plt.ylabel("Number of Customers")

    plt.tight_layout()
    plt.show()
```

```
============================================================
PREDICTED CHURN DISTRIBUTION
============================================================
Predicted_Churn
False    2
True     2
Name: count, dtype: int64
```

